Software Design Document (SDD)

CRASH SENSE: AI-Based Application Crash Prediction & Prevention System

Document Control	**Group 8** **Muhammed Shafi A S** **Hasna Sherin C K** **Ananthu A R** **Madhav R**
Version	1.0
Status	**Initial Draft**
Date	January 14, 2026
Project Name	**CRASH SENSE**
Prepared For	Mini-Project

Table of Contents

Section No	Title	Page Number
1	**INTRODUCTION**	**3**
1.1	Purpose	3
1.2	Scope	3
1.3	Overview	4
1.4	Reference Material	4
1.5	Definitions and Acronyms	4
2	**SYSTEM OVERVIEW**	**4**
3	**SYSTEM ARCHITECTURE**	**4**
3.1	Architectural Design	4
3.2	Decomposition Description	5
3.3	Design Rationale	6
4	**DATA DESIGN**	**7**
4.1	Data Description	7
4.2	Data Dictionary	8
5	**UNIFIED MODELLING LANGUAGE (UML) DIAGRAM**	**8**
5.1	Use Case Diagram	8
6	**DATA FLOW DIAGRAMS**	**9**
7	**COMPONENT DESIGN**	**10**
7.1	DataCollector Component	11
7.2	ModelEngine Component	11
7.3	RemediationManager Component	11
8	**HUMAN INTERFACE DESIGN**	**11**
8.1	Overview of User Interface	11
8.2	Screen Images	11
8.3	Screen Objects and Actions	15
9	**REQUIREMENTS MATRIX**	**15**

1. INTRODUCTION

1.1 Purpose

This Software Design Document (SDD) describes the architecture and system design of CRASH SENSE. It is intended for the development team and project stakeholders to understand the implementation details of the crash prediction system.

1.2 Scope

The **Crash Sense: AI-Based Application Crash Prediction & Prevention System** aims to overcome the limitations of existing crash detection tools by implementing a multi-modal, predictive, and explainable AI framework.

1.2.1 In-Scope Functions (Deliverables)

The system will encompass the following key functionalities:

- **Multi-Source Data Integration:** Unlike systems that rely solely on one data type, this project will simultaneously ingest and analyze two distinct data sources:
  - **System/Application Logs:** Text-based error logs and event streams.
  - **Runtime Performance Metrics:** Numerical data such as CPU usage, memory consumption, and I/O rates.
- **Advanced Data Synchronization:** The system will implement a preprocessing pipeline to align unstructured log data with structured performance metrics based on timestamps, ensuring a unified dataset for analysis.
- **Predictive Anomaly Detection:** The system will go beyond simple "post-failure" reporting. It will utilize Machine Learning models to identify pre-crash patterns and forecast potential failures *before* they result in a system shutdown.
- **Explainable Root Cause Analysis (XAI):** To address the "black box" issue of current models, the system will provide detailed explanations for every alert. This includes highlighting the specific log lines or metric spikes that contributed to the crash prediction.
- **Robust False Positive Reduction:** The model will be trained on diverse datasets to improve generalization and reduce false alarms, ensuring that alerts sent to developers are actionable and accurate.

1.2.2 Out-of-Scope (Exclusions)

To ensure the project remains feasible within the timeline, the following are excluded:

- **Automatic Remediation:** The system will detect and explain the crash, but it will *not* attempt to automatically patch the code or restart the server.
- **Hardware-Level Diagnostics:** The scope is limited to software-level logs and metrics; physical hardware faults (e.g., disk failure, overheating) are not monitored unless reflected in OS logs.
- **Real-Time Distributed Tracing:** The system focuses on single-node application monitoring rather than complex microservice distributed tracing across multiple servers.

1.3 Overview

This document is organized into eight sections. Section 2 provides a general system overview. Section 3 details the architectural design. Section 4 covers data structures. Section 5 details component design algorithms. Section 6 presents the Human Interface Design (UI/UX). Section 7 provides the Requirements Matrix.

1.4 Reference Material

- Crash_Sense_SRS_Version1.1.pdf (Software Requirements Specification)
- IEEE Std 1016-2009 (Standard for Information Technology—System Design)

1.5 Definitions and Acronyms

- **SDD:** Software Design Document
- **SRS:** Software Requirements Specification
- **HITL:** Human-in-the-Loop (User permission required for actions)
- **RCA:** Root Cause Analysis

---

2. SYSTEM OVERVIEW

The CRASH SENSE system operates as an active guardian rather than a passive monitor. It functions as a middleware agent sitting between the Linux Operating System and the Target Application.

The system continuously polls hardware metrics and parses application logs. When the AI models detect anomaly patterns indicative of an impending crash (e.g., memory leaks or thread exhaustion), the system triggers an alert. The system then presents a diagnosis and a recommended remediation action (e.g., "Restart Service") to the administrator. Upon approval, the system executes the fix via shell scripts to restore stability.

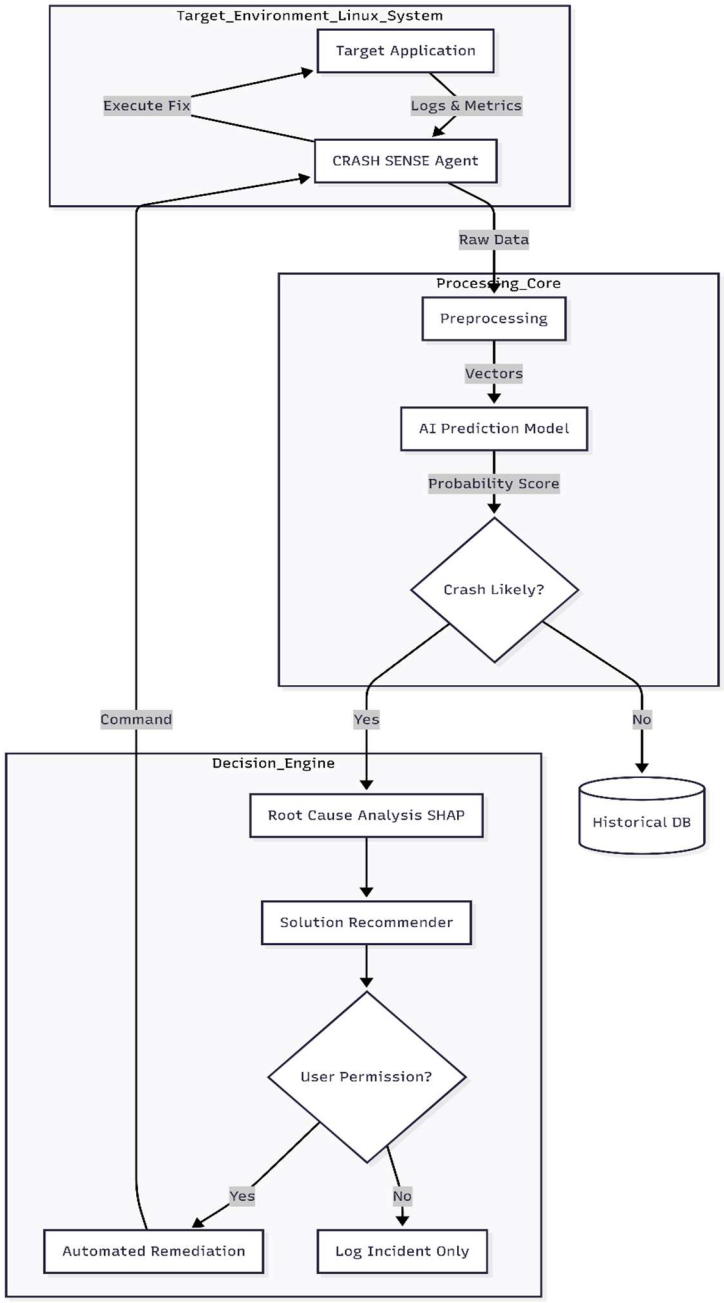
3. SYSTEM ARCHITECTURE

3.1 Architectural Design

The system utilizes a **Local Agent-Based Architecture** combined with a **Data Processing Pipeline**. The architecture is modular, separating data collection, processing, and visualization to ensure scalability and low latency.

High-Level Modules:

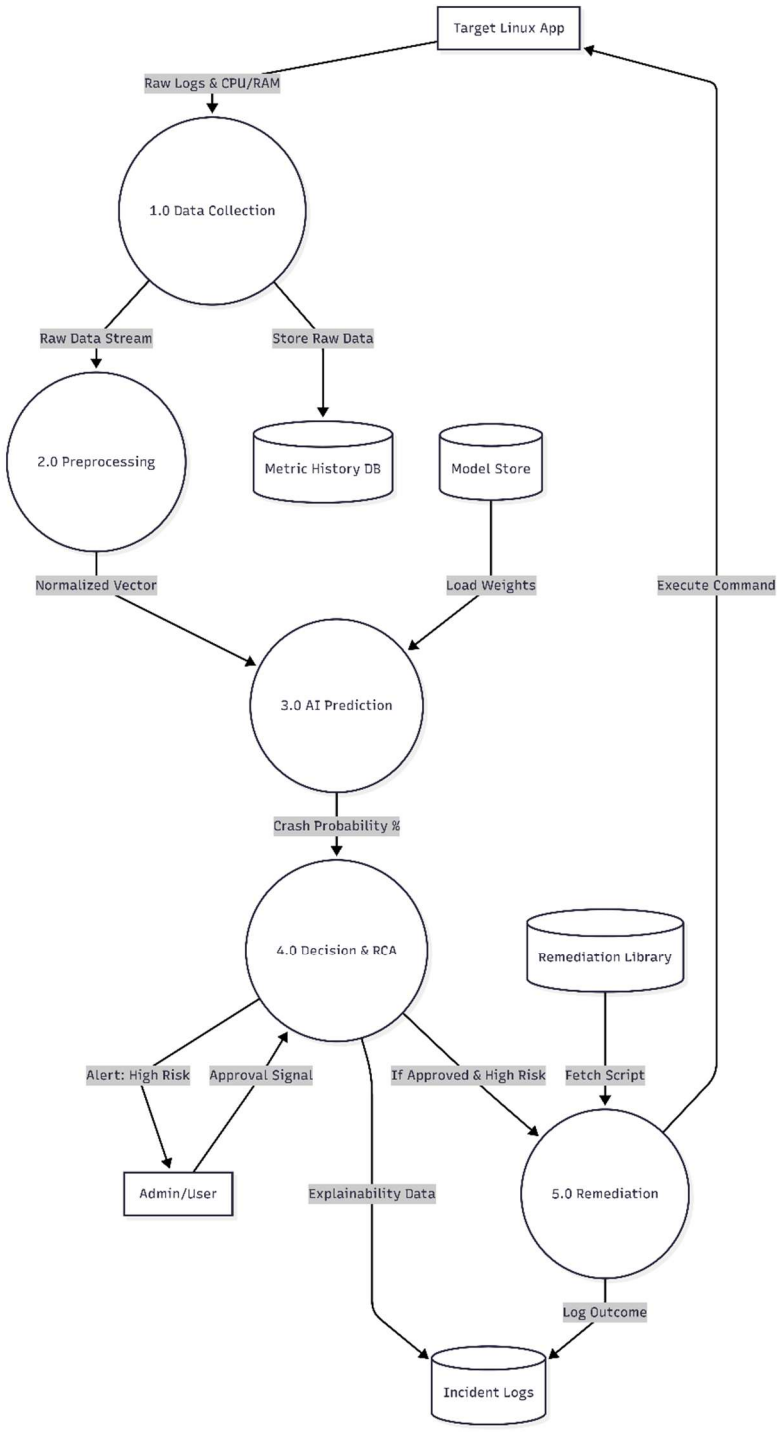
1. **Target Environment (Linux):** Hosts the Application and the CRASH SENSE Agent.
2. **Processing Core:** Handles data normalization, vectorization, and AI inference.
3. **Decision Engine:** Manages logic for alerts, RCA, and remediation execution.
4. **Presentation Layer (UI):** A web-based dashboard for user interaction.



3.2 Decomposition Description

The system is decomposed into four primary subsystems:

1. **Ingestion Subsystem (DataCollector):** Collects raw data from the OS kernel (CPU/RAM) and tails log files.
2. **Processing Subsystem (ModelEngine):** Converts raw data into feature vectors and generates probability scores.
3. **Decision & Remediation Subsystem:** Interprets probabilities, queries the "Remediation DB" for solutions, and executes scripts.
4. **Visualization Subsystem:** Renders the dashboard and alerts.



3.3 Design Rationale

The Local Agent-Based Architecture was selected over a cloud-based remote monitoring architecture to satisfy NFR-01 (Performance).

- **Critical Issue:** Crash prediction requires extremely low latency (<200ms).
- **Trade-off:** Running the AI model locally consumes some client resources (<5% CPU), but it eliminates network latency, ensuring the system can react fast enough to prevent a crash.

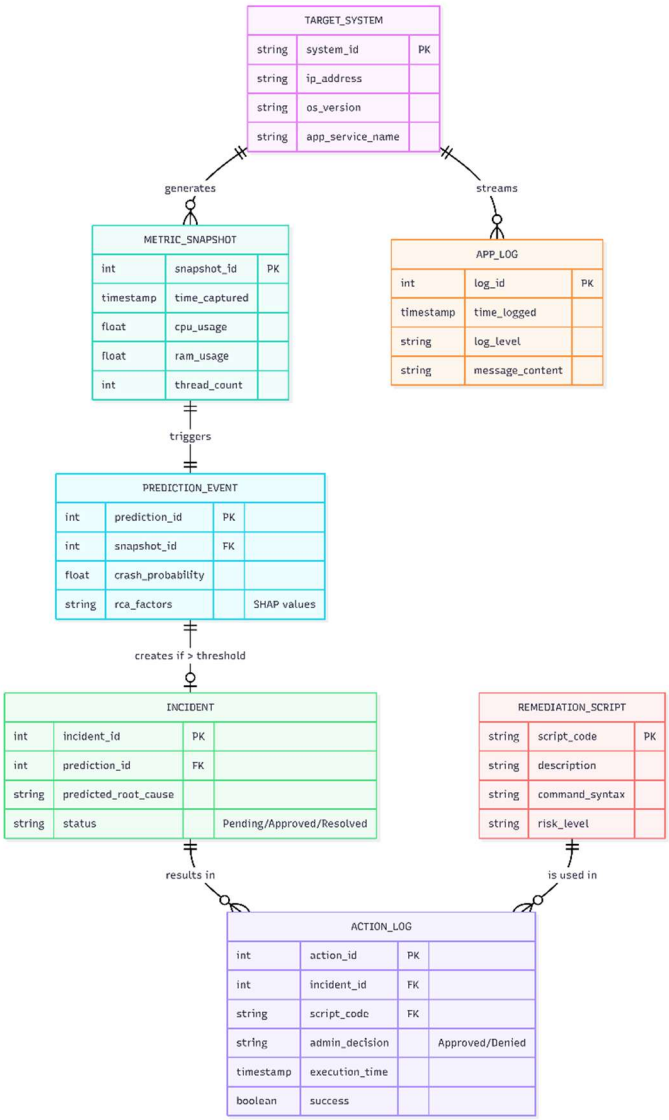
---

4. DATA DESIGN

4.1 Data Description

The system handles two primary streams of data:

1. **Time-Series Metrics:** Structured numerical data (CPU Load, RAM Usage, Disk I/O) collected every 500ms.
2. **Unstructured Logs:** Textual log lines generated by the target application.



4.2 Data Dictionary

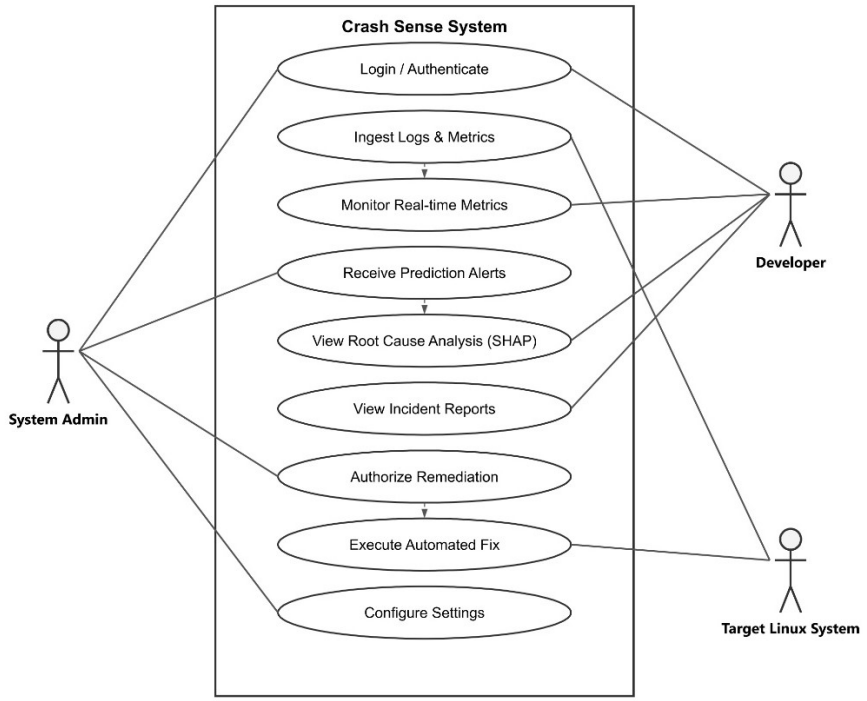
Data Entity	Type	Description
crash_prob	Float	Predicted probability of failure (0.00 - 1.00).
incident_id	UUID	Unique identifier for a detected crash event.
metric_cpu	Float	CPU usage percentage (0.0 - 100.0).
remediation_script	String	Path to the Bash/Python script (e.g., /scripts/restart.sh).

---

5. UNIFIED MODELLING LANGUAGE (UML) DIAGRAM

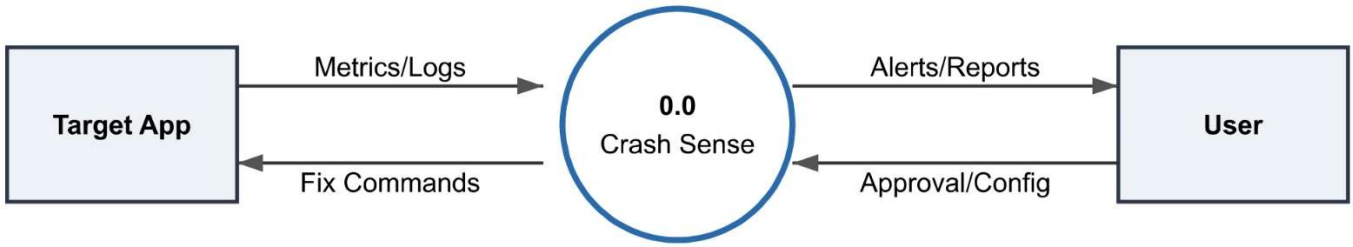
5.1 Use Case Diagram

This diagram defines the interactions between the users (Admin, Developer, Target Linux System) and the Crash Sense system features described in Sections 2.2 and 4 (In SRS document).

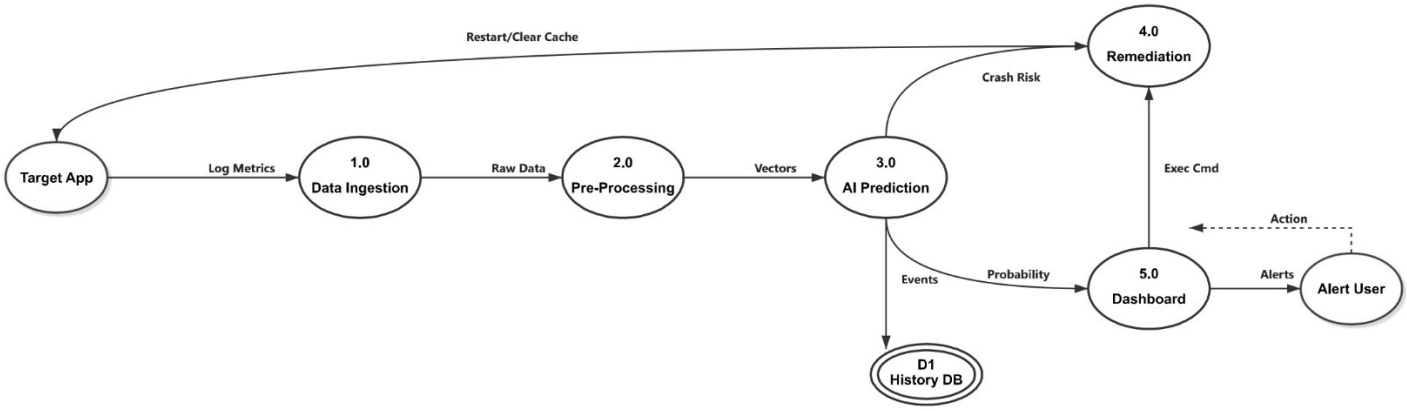


6. DATA FLOW DIAGRAMS

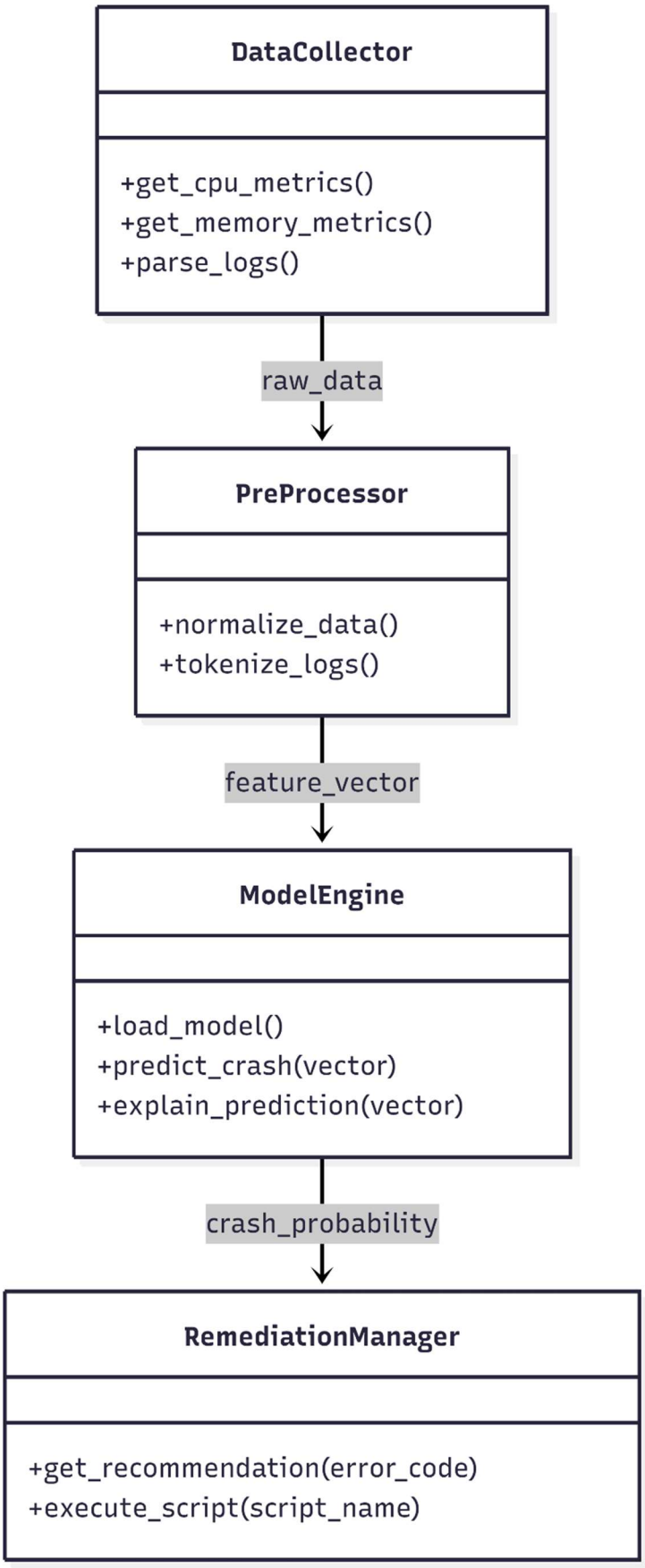
6.1 Level 0



6.2 Level 1



7. COMPONENT DESIGN



7.1 DataCollector Component

- **Function:** collect_metrics()
- **Logic:** Polls psutil library for CPU/RAM and watchdog for new log lines. Returns a dictionary object.

7.2 ModelEngine Component

- **Function:** predict_crash(data)
- **Logic:** Normalizes input data and feeds it into the pre-trained LSTM network. Returns a float probability.

7.3 RemediationManager Component

- **Function:** execute_fix(script_name)
- **Logic:** Checks if user_permission == True. If yes, uses subprocess.run() to execute the script. If no, logs "Action Denied".

---

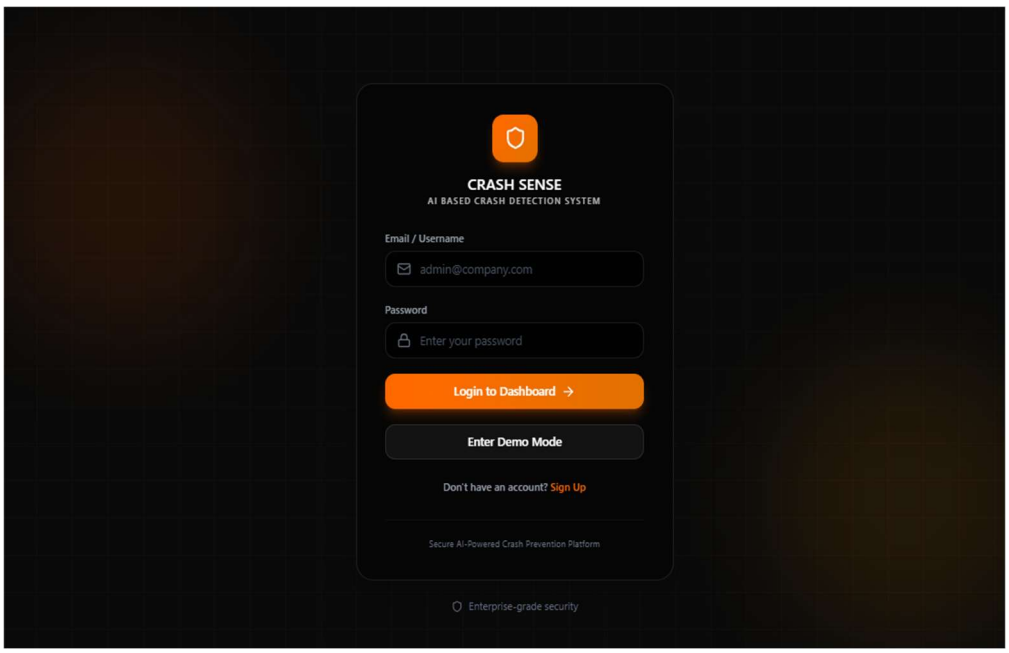
8. HUMAN INTERFACE DESIGN

8.1 Overview of User Interface

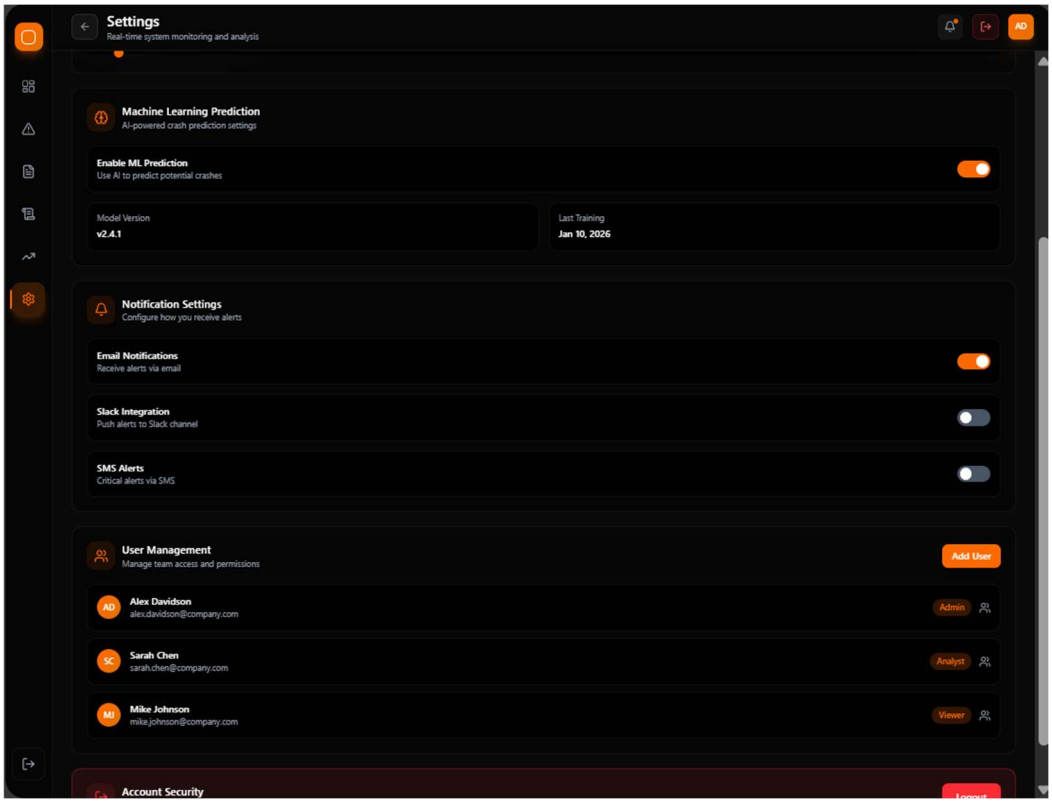
The user interface is a web-based dashboard designed with a "Cyber-Security / SOC" aesthetic (Dark Mode). It allows the user to monitor system health in real-time, view crash predictions, and authorize remediation actions.

8.2 Screen Images

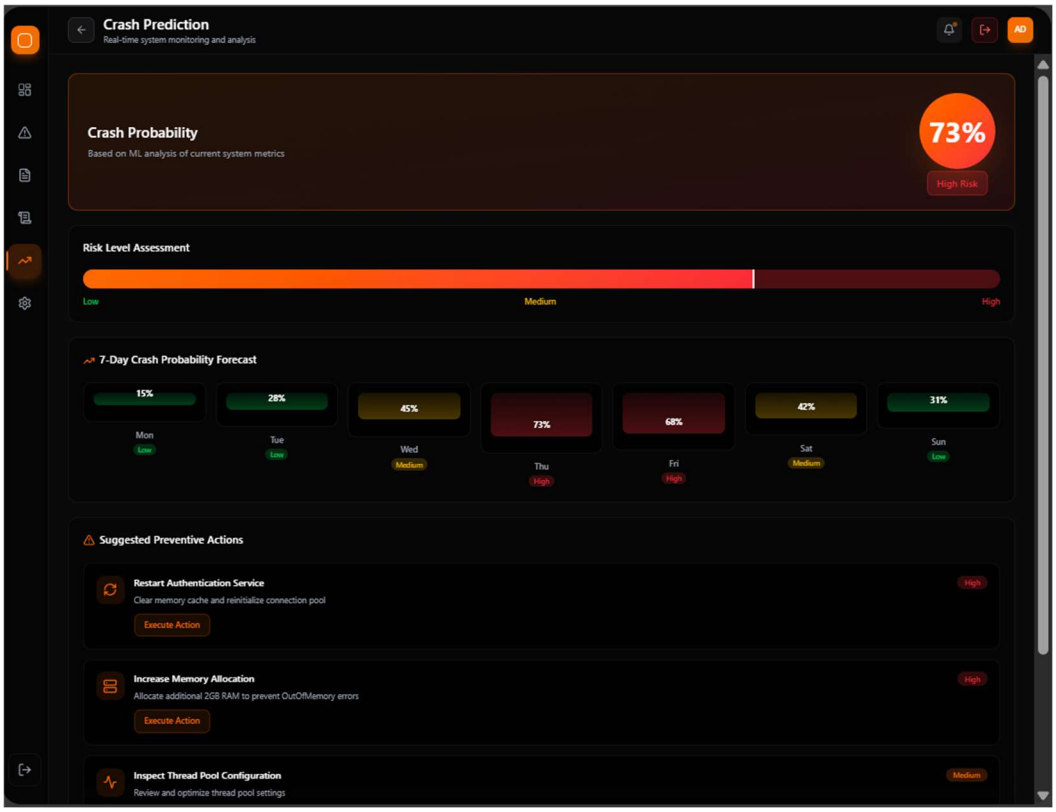
Login screen



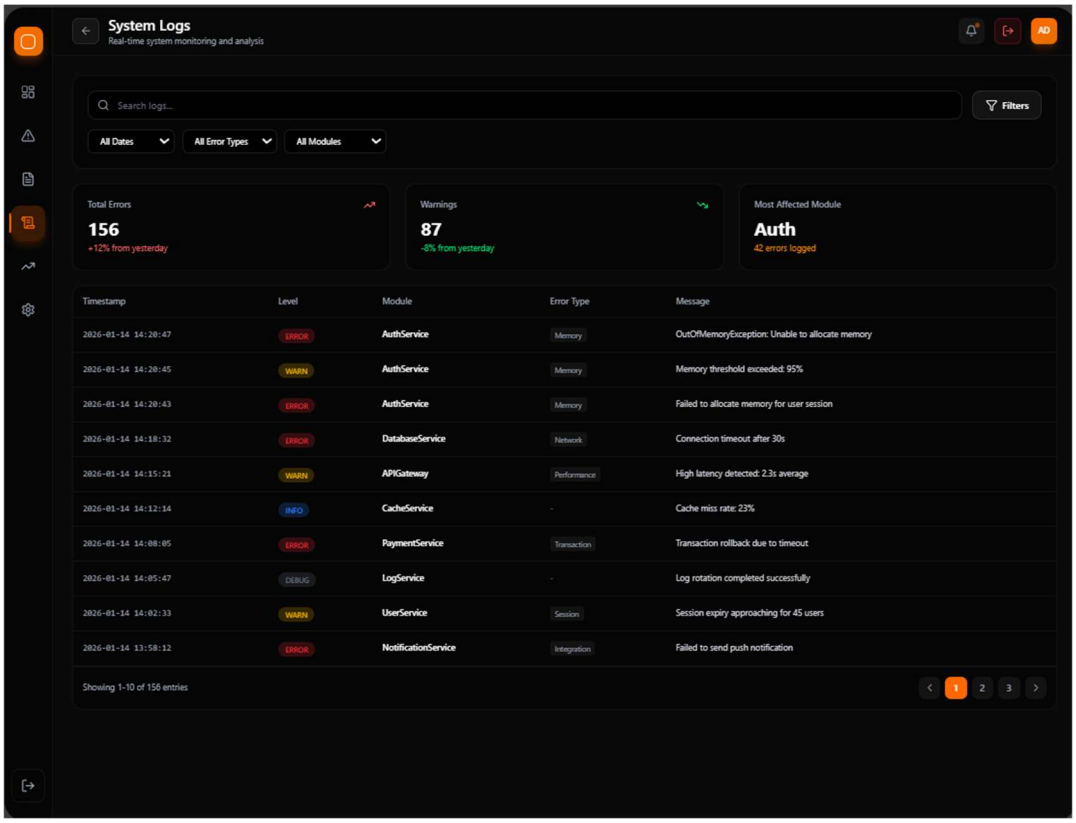
Settings



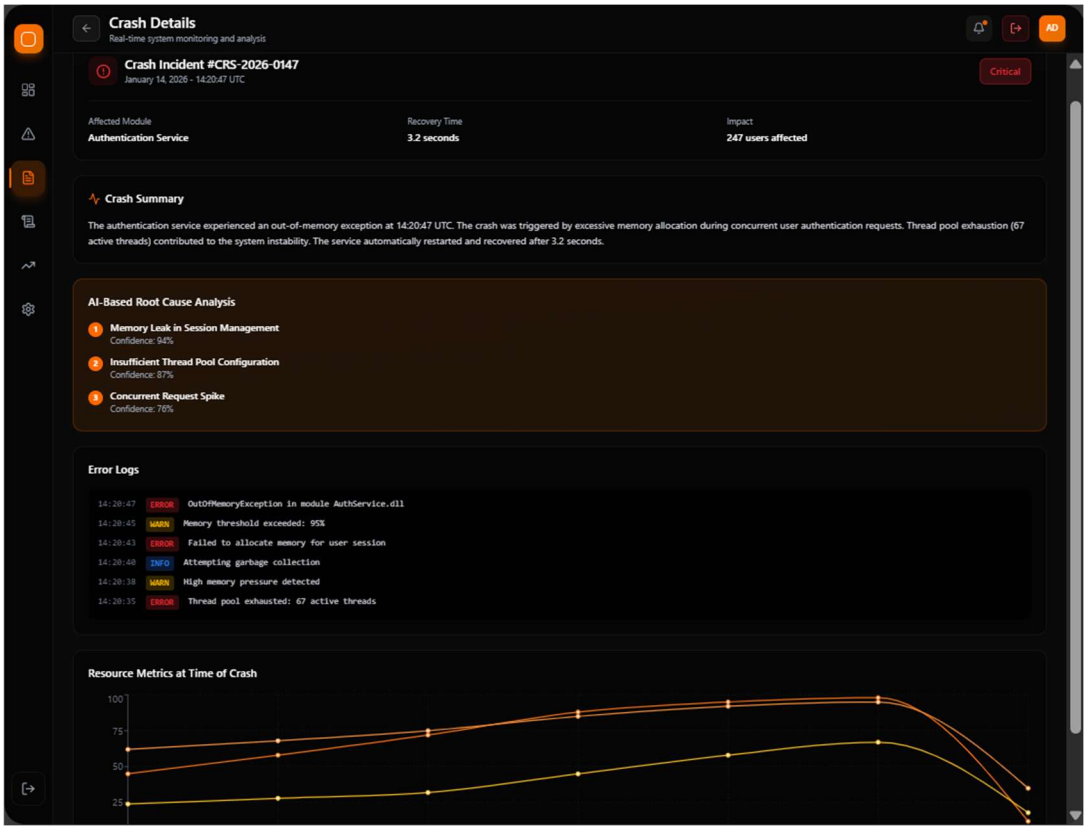
Crash Prediction



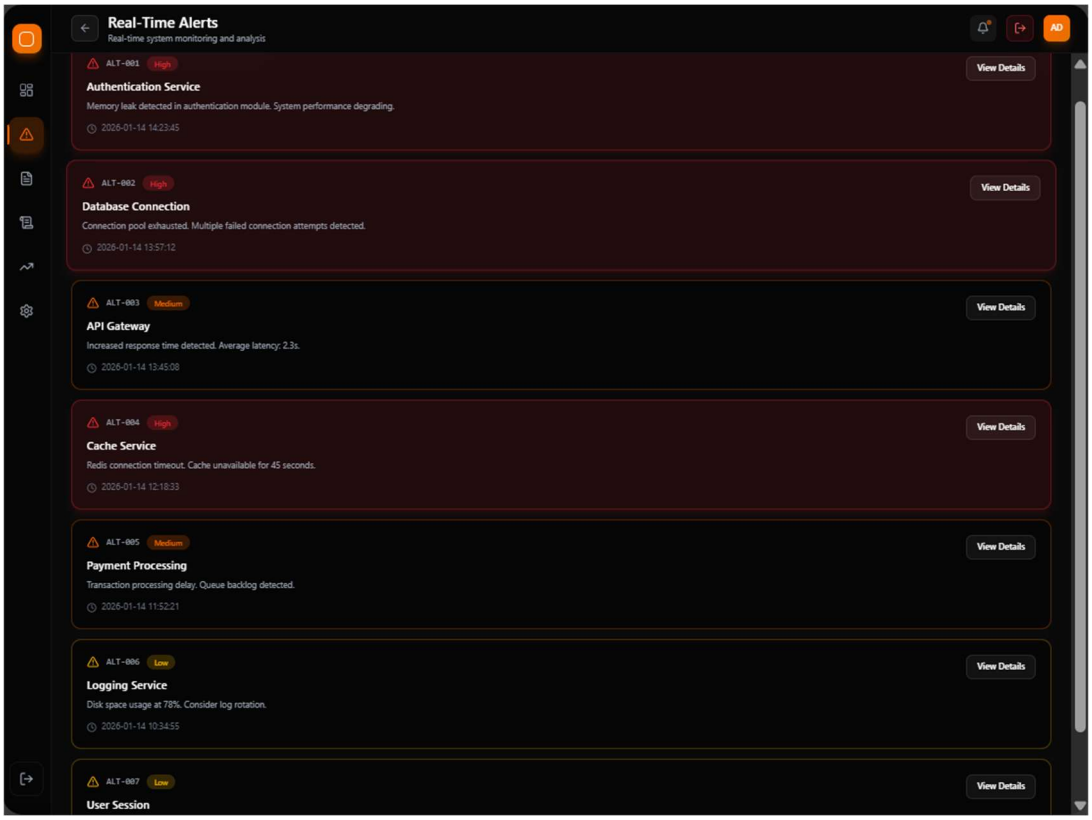
Systems Logs



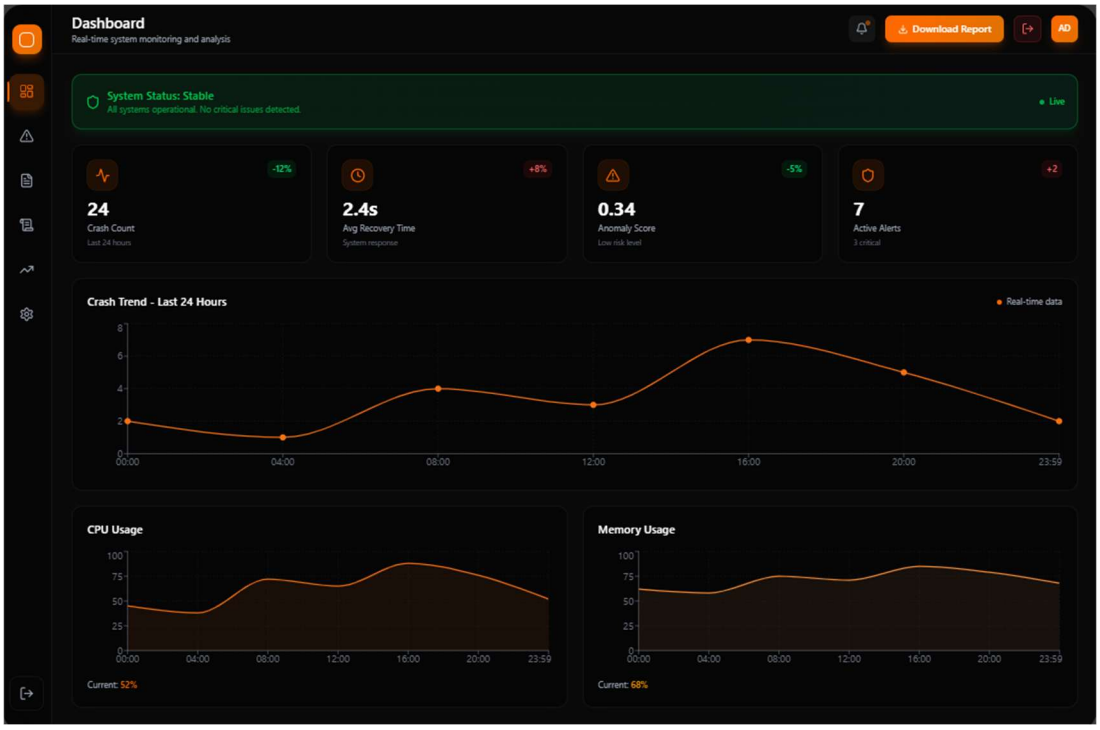
Crash Details



Real-Time Alerts



Dashboard



8.3 Screen Objects and Actions

- **"Execute Action" Button:** Triggers remediation scripts.
- **"View Details" Button:** Navigates to the specific incident report.
- **"Enable ML Prediction" Toggle:** Activates the AI model.

---

9. REQUIREMENTS MATRIX

Requirement ID	Description	Verification Status	Test Cases & Inputs
FR-01	Collect CPU, Memory, and Disk I/O metrics every 500ms from the host Linux system.	☐	• TC-01: Run collector for 10s. *Input:* Host system active. *Expected:* 20 distinct metric records (CPU, Mem, Disk) with ~500ms timestamps.
FR-02	Parse application-specific logs in real-time.	☐	• TC-02: Inject error log. *Input:* Write [ERROR] DB Connection Timeout to log file. *Expected:* Parser event triggered within <100ms.
FR-03	Predict a crash at least 30 seconds before occurrence with 85% accuracy.	☐	• TC-03: Simulate crash pattern. *Input:* Replay 'Memory Leak' dataset sequence. *Expected:* Prediction alert generated 30s before dataset end.
FR-04	Identify the "Leading Indicator" (e.g., Memory spike) using SHAP values.	☐	• TC-04: Check feature attribution. *Input:* High CPU load scenario.

			Expected: SHAP analysis highlights 'CPU_Usage' as top factor.
FR-05	Maintain a database of "Crash Signatures" mapped to "Recommended Actions".	☐	• TC-05: Verify Signature Mapping. *Input:* Query DB for signature "OOM Error". *Expected:* Returns action "Clear Temp Cache" or "Restart App".
FR-06	Present the recommended action to the user via a prompt.	☐	• TC-06: Trigger maintenance alert. *Input:* Threshold breach event. *Expected:* UI displays prompt with "Restart Service" recommendation.
FR-07	Execute the remediation script (Bash/Python) only upon receiving boolean True (Permission Granted).	☐	• TC-07: Test User Permission. *Input:* Click 'Deny' on remediation prompt. *Expected:* No script execution; system logs "User Denied".
FR-08	Verify if the application stabilized after the action.	☐	• TC-08: Post-Action Validation. *Input:* Execute 'Restart' action. *Expected:* Health metrics return to green zone (CPU < 50%) for >2 mins.